

דוח שינויים מלא

מהענף `smdp` של upstream אל הענף `smdp-dev` — מה שונה, ולמה

פרויקט: `sched-rl-gym` — סביבת RL לתזמון משימות ב-HPC (fork של `renatolfc/sched-rl-gym`)

ריפוי: `github.com/RoeHaim/sched-rl-gym`, ענף עבודה: `smdp-dev`

טווח: `upstream/smdp...smdp-dev` — 7 קומיטים, כולל מיזוג המודרניזציה של `master`

תאריך: 18 ביולי 2026

מצב אימות: 181/181 טסטים עוברים · `eval_baselines` רץ · Rust backend פעיל

תוכן עניינים

- 1 תקציר מנהלים
- 2 רקע: מצב הענפים לפני העבודה
- 3 קומיטי התשתית המוקדמים
- 4 המיזוג עם master — העיקרון המנחה ופתרון הקונפליקטים קובץ־קובץ
- 5 הפורט ל-API של gymnasium
- 6 תיקוני הפוסט־מיזוג: התאמת הטסטים החדשים
- 7 שינויי הסביבה: uv, Python 3.13, Rust, vendoring
- 8 אימות התוצאה
- 9 נקודות פתוחות והמלצות

1. תקציר מנהלים

המטרה: לעבוד על מחקר ה-SMDP על גבי סביבה מעודכנת. הבעיה: ענף המחקר `smdp` נשאר על סטאק ישן (`gym` 0.21, `Python 3.10`, `setup.py`, התקנה שדורשת נעילות `pip` היסטוריות), בעוד שה-`master` של `upstream` עבר מודרניזציה מקיפה: `gymnasium`, פקג'ינג מבוסס `uv` + `pyproject`, הרחבת ביצועים ב-`Rust`, ועשרות תיקוני באגים ואופטימיזציות.

הפתרון שבוצע: **מיזוג של `upstream/master` לתוך קו ה-SMDP** תחת עיקרון מנחה אחד — **סמנטיקת המחקר של ה-SMDP נשמרת אחד־לאחד; ה-API, הפקג'ינג והביצועים מאומצים מ-`master`**. בנוסף בוצעו התאמות לטסטים החדשים של `master` (שהניחו ייצוג `state` ללא פיצ'רי ה-SMDP), תוקנו סקריפטי ההערכה, והוקמה סביבה חדשה ורפרודוצבילית.

שורה תחתונה

כל 181 הטסטים עוברים (כולל 98 הטסטים ההיסטוריים של ענף ה-`smdp`), התנהגות הסימולציה וייצוג ה-`state` של ה-SMDP לא השתנו, והסביבה כולה מותקנת בפקודה אחת: `uv sync --extra test --group rl`.

2. רקע: מצב הענפים לפני העבודה

2.1 שלושת הקווים

ענף	מה יש בו	סטאק
upstream/smdp	קו המחקר: תמיכה ב-SMDP (סימולציה מבוססת-אירועים, תגמול מהוון γ , state time offsets ב-state, פיצ'ר <code>can_schedule_now</code> , טעינת SWF, ועוד)	setup.py + gym 0.21 Python, requirements.txt 3.10
upstream/master	מודרניזציה: פורט ל-gymnasium + תיקון 9 באגים, Rust backend (PyO3) ל-Heap/IntervalTree/ResourcePool של ה-bitmask scheduler, מנוע replay, טסטים חדשים	gymnasium ≥ 1.0 , pyproject.toml + uv + Python ≥ 3.10 , maturin
smdp-dev (שלי)	smdp + קומיטי תשתית שלי (פירוט בפרק 3), ועכשיו — גם המיזוג עם master	אחרי המיזוג: הסטאק המודרני במלואו

2.2 עומק ההתפצלות

בזמן המיזוג, master החזיק **35 קומיטים** שלא היו ב-smdp, ו-smdp החזיק **42 קומיטים** שלא היו ב-master. שני הצדדים נגעו באותם קבצי ליבה (scheduler, envs, job) — ולכן מיזוג אוטומטי לא היה אפשרי: git דיווח על **13 קבצים בקונפליקט** (12 קונפליקטי תוכן + מחיקת setup.py מול שינוי מקומי בו).

2.3 למה בכלל למזג ולא להישאר על הישן?

- סביבה:** הסטאק הישן דרש טריקים שבירים — `pip < 24.1` בגלל metadata פגום של gym 0.21, `setuptools 65`, `numpy 1.23`. כל מכונה חדשה הייתה פרויקט התקנה בפני עצמו.
- אבטחה ותחזוקה:** העדכון האחרון של upstream היה שדרוג dependencies מטעמי אבטחה — רלוונטי רק לסטאק החדש (uv.lock).
- ביצועים:** אימון RL מריץ מיליוני צעדי סימולציה; האופטימיזציות של master (Rust heap, bitmask scheduler, caching) מקצרות את זמן האימון ישירות.
- עתיד:** כל עדכון עתידי של upstream יגיע על הסטאק החדש. מיזוג עכשיו = מיזוגים זולים בהמשך.

3. קומיטי התשתית המוקדמים

ארבעה קומיטים שקדמו למיזוג (עבודה שנעשתה קודם ושומרה מהענף הישן `decouple-simulator-mdp-smdp` שנמחק; גיבוי מלא שלו קיים מקומית ב־ `backup/decouple-simulator-mdp-smdp`):

קומיט	מה	למה
902df95	<code>deepm-agent.py</code> חוצה־פלטפורמות: <code>guard</code> על <code>os.getuid()</code> (לא קיים ב־Windows) עם <code>fallback</code> ל־ <code>tempfile.gettempdir()</code> , ויצירה אוטומטית של תיקיית <code>checkpoint</code> .	סקריפט האימון קרס ב־Windows על שורת ה־TMPDIR עוד לפני שהתחיל. העבודה נעשית על מכונת Windows.
2a995e0	<code>eval_baselines.py</code> (הרצת SJF/Random עם תגמול מצטבר ו־slowdown ממוצע) ו־ <code>eval_policy.py</code> (טעינת <code>checkpoint</code> והשוואה מול ההיוריסטיקות).	הסקריפט המקורי (<code>sjf-agent.py</code>) דרש <code>rendering</code> ; למחקר צריך מדידה שקטה, רבת אפיזודות, על אותם <code>seeds</code> — התשתית להשוואת MDP מול SMDP.
7e61f22	תיקיית <code>project/</code> (יומן התקדמות, מדריך, לוגים) והרחבת <code>.gitignore</code> (<code>runs</code> , <code>pth.*</code> , <code>venv</code> , ארטיפקטים של C).	ריכוז חומרי הפרויקט לצד הקוד, בהפרדה מקבצי המקור של <code>upstream</code> .
08ce20f	<code>project/ENVIRONMENT.md</code> (צעדי ההקמה המדויקים על Windows בסטאק הישן), <code>requirements-lock.txt</code> קפוא, <code>test/LANL-CM5-1994-4.1-cln.swf.gz</code> <code>trace</code> ־ה ו־ <code>gitattributes</code> שמונע הסבת CRLF לקבצי <code>trace</code> .	רפרודוציביליות: התקנת הסטאק הישן הייתה מוקש (ראו 2.3). ה־ <code>trace</code> נדרש ל־ <code>TestSwfGenerator</code> ולא הגיע עם הריפו.

שני הקבצים האחרונים רלוונטיים היום בעיקר כתיעוד היסטורי — `ENVIRONMENT.md` נכתב מחדש אחרי המיזוג (פרק 7), אבל ה־`lockfile` הישן נשמר למקרה שיהיה צורך לשחזר ריצות ישנות בדיוק.

4.1 העיקרון המנחה

הכלל שהכריע כל קונפליקט

כשהצדדים התנגשו על סמנטיקה של הסימולציה או של ייצוג ה-state — ניצח צד ה-SMDP, גם כשהקוד של master היה "נקי" או מהיר יותר. כשההתנגשות הייתה על API, סגנון, פקג'ינג או ביצועים שאינם משנים תוצאה — נוצח צד master. הסיבה: כל שינוי סמנטי שקט היה מנתק את ההשוואה מול ניסויים קודמים והופך תוצאות עתידיות ללא-ברות-השוואה.

4.2 מפת הקונפליקטים

קובץ	קונפליקטים	הכרעה עיקרית
shedgym/job.py	2	איחוד: dataclass של master + שדה SMDP
shedgym/workload/swf_parser.py	2	צד SMDP (פרסינג סלחני)
shedgym/envs/workload.py	1	צד SMDP (swf + seed) + תיקון באג
shedgym/envs/simulator.py	2	איחוד טיפוסים בתחביר מודרני
shedgym/envs/base.py	3	צד SMDP + פורט gymnasium + גשר seed
shedgym/envs/deeprm_env.py	2	לוגיקת תגמול SMDP + tuple-5
shedgym/envs/compact_env.py	8	שכתוב מלא: state של SMDP, API של gymnasium
shedgym/scheduler/scheduler.py	1	מסלול SMDP נשמר; מסלול MDP קיבל את המהיר של master
shedgym/test_shedgym.py	3	ציפיות הענף שלנו, API חדש
deeprm-agent.py	1	תאימות Windows נשמרה
shedgym/__init__.py , .gitignore	3+1	איחודים קטנים
setup.py	מחיקה/שינוי	נמחק — הוחלף ב-pyproject.toml

4.3 פירוט קובץ קובץ

schedgym/job.py – JobState

הפך את JobState מ-namedtuple ל-dataclass קפוא עם 7 שדות (בטוח יותר, עם ברירות מחדל -1).
הוסיף ל-namedtuple שדה שמיני — `can_schedule_now` — פיצ'ר שמסמן לסוכן אם הג'וב יכול להתחיל מיד.

ההכרעה: איחוד — הצורה של master, התוכן של smdp:

```
@dataclass(frozen=True)
class JobState:
    submission_time: int = -1
    requested_time: int = -1
    requested_memory: int = -1
    requested_processors: int = -1
    queue_size: int = -1
    queued_work: int = -1
    free_processors: int = -1
    can_schedule_now: int = -1      # <-- the SMDP-side 8th field

    def __len__(self):
        return 8                  # was 7 on master
```

למה: ה-scheduler בענף שלנו ממלא את השדה (ב-`Scheduler.state`), ומחיקתו הייתה שוברת את ייצוג הג'ובים. צורת ה-dataclass של master עדיפה על namedtuple (מפורשת, ברירות מחדל) בלי שום שינוי התנהגות. בנוסף נשמר תיקון קטן מהענף שלנו: פסיק שחסר ב-`__str__` של Job בין `memory` ל-`duration`.

schedgym/workload/swf_parser.py

מדלג על כל שורה עם ; **ודוחה** ג'ובים פגומים (מעבדים/זמן/זיכרון לא-חיוביים) עם אזהרה.
קורא מהכותרות את `MaxNodes / MaxProcs` **ומתקן** ג'ובים פגומים במקום לזרוק אותם:

```
if line.startswith(";"):
    if line.startswith("; MaxNodes:"):
        max_nodes = parse_int(line)
    elif line.startswith("; MaxProcs:"):
        max_procs = parse_int(line)
    ...
if job.requested_processors < 1: job.requested_processors = 1
if job.requested_memory < 1: job.requested_memory = 1
if job.execution_time < 1: job.execution_time = 1
if job.submission_time < 0: job.submission_time = 0
```

ההכרעה: צד smdp במלואו (בסגנון המרכאות של master). **למה:**

- קריאת הכותרות היא **תלות קשיחה**: הקוד שממשיך אחרי הקונפליקט משתמש ב-`max_nodes / max_procs` כדי לטעון trace בלי לציין ידנית את מספר המעבדים ("make SWF loader not require number of processors"). בחירת צד master הייתה מקריסה אותו.
- הפרסינג הסלחני הוא החלטה מחקרית מהענף ("be less strict on the jobs we accept to parse"): מה-trace של LANL-CM5 מתקבלים **122,060** ג'ובים, מול 122,052 אצל master. שינוי היה משנה את הרכב ה-workload בכל הניסויים ושובר השוואות לריצות עבר.

`build()` : מקבל `random_seed` ומכיר סוג `workload` שלישי — `swf` (`SwfWorkloadGenerator`).
`seed`, בלי `swf`. **ההכרעה**: צד `smdp` — שני הפיצ'רים (זריעה דטרמיניסטית וטעינת `traces` אמיתיים) הם תשתית מחקר פעילה, והמחלקות עצמן שרדו את המיזוג האוטומטי ממילא.

תיקון אגבי באג סמוי ב־`SwfWorkloadGenerator`: כשמועבר `length=None` הקוד השווה `int < None` וקרט. הוחלף בשימוש ב־`self.length` המחושב, והאנוטציה עודכנה מ־ `Optional[int]` (שבכלל לא היה מיובא) ל־`int` | `None` המודרני:

```
self.length = len(self.orig_swf) if length is None else length
if self.length < len(self.orig_swf):
    start = random.randint(0, len(self.orig_swf) - self.length)
```

שני קונפליקטים טכניים על טיפוס האיחוד של הגנרטורים. `master` השמיט את `SwfWorkloadGenerator` (לא קיים אצלו); אצלנו הוא חובה. אוחד בתחביר המודרני:

```
WorkloadGeneratorType = (
    DeepRmWorkloadGenerator | SyntheticWorkloadGenerator | SwfWorkloadGenerator
)
```

הקובץ שבו נפגשים ה-API החדש והסמנטיקה הישנה. שלוש הכרעות:

(א) **מנגנון הזריעה של הענף נשמר.** master מחק את `seed()` (gymnasium עבר ל-`reset(seed=...)`). אצלנו הזריעה מחוברת לבניית ה-workload (`build_workload(config, self.random_seed[0])`) — בלעדיה אין רפרודוציביליות של הניסויים. נשמר, כולל ה-`tolerance_factor` (מכפיל שמונע סימולציות אינסופיות על traces) שמאסטר לא מכיר.

(ב) **גשר בין שני העולמות.** נוסף חיבור כך שהקונבנציה החדשה של gymnasium מפעילה את המנגנון הישן — מי שקורא `reset(seed=N)` מקבל דטרמיניזם מלא, ומי שלא מעביר seed מקבל את ההתנהגות הישנה בדיוק:

```
def reset(self, *, seed=None, options=None) -> tuple[np.ndarray, dict]:
    super().reset(seed=seed, options=options)
    if seed is not None:
        self.seed(seed)                # bridge: gymnasium -> legacy seeding
    ...
    wl = build_workload(
        self.workload_config,
        self.random_seed[0] if (seed is not None or not self.has_reset) else None,
    )
    ...
    return self.state, {}              # gymnasium: (obs, info)
```

(ג) **הדגל smdp ממשיך לזרום ל-scheduler.** `_render_state` ו-`state` קוראים

`scheduler.state(time_horizon, job_slots, self.smdp)` — הפרמטר השלישי לא קיים אצל master ונשמר מהענף.

שימו לב לפרט ב-(ב): master בכלל לא העביר seed ל-workload; אצלנו הוא מועבר בבנייה הראשונה או כשניתן seed מפורש — בדיוק כסמנטיקת הענף המקורית ("refrain from fixing random seed by default").

שני קונפליקטים; המהותי הוא חישוב התגמול המהוון של ה-SMDP. master מאפס תמיד את `rewards[0]`; הענף שלנו מאפס רק כשיש יותר מצעד ביניים אחד — זה התיקון "only zero reward when jobs are scheduled" מהענף:

```
if not done and self.smdp and any(intermediate):
    rewards = [self.compute_reward(js) for js in intermediate]
    if len(rewards) > 1:                # smdp-branch guard (master zeroed always)
        rewards[0] = 0
    reward = (self.gamma ** np.arange(len(intermediate))).dot(rewards)

return (self.state, reward, done, False, self.stats if done else {}) # 5-tuple
```

למה: ההצמדה לענף שומרת על פונקציית התגמול שאיתה נמדדו התוצאות עד כה. שורת ההחזרה עברה לפורמט gymnasium (`truncated=False, terminated=done` — כבחירת master).

8 קונפליקטים שנבעו מכך ששני הצדדים שכתבו את אותה מתודה — `state` property — לכיוונים שונים לגמרי:

smdp		master
מבנה ה-state scheduler	[processors, memory]	[time offsets, processors, memory] — "החלון לעתיד" של ה-SMDP
סקלת זמן	לינארית	לוגריתמית (log-transform על offsets, remaining work, time left)
מה הסוכן רואה על ג'ובים רצים	execution_time (זמן ריצה אמיתי)	requested_time בלבד — הסוכן לא אמור לדעת זמני ריצה אמיתיים
מימוש	buffer קדם-מוקצה (אופטימיזציה)	הקצאה חדשה בכל קריאה

ההכרעה: המיזוג האוטומטי ערבב את שתי הגישות לקוד לא-קוהרנטי, ולכן state property וסביבתו **שוחזרו מילה-במילה מגרסת הענף** (מתוך `git show smdp:...`), עם שני שינויים בלבד: פורמט ההחזרה של `reset / step` עבר ל-`gymnasium`, וה-`spaces` עברו ל-`gymnasium.spaces`.

למה ויתרנו על ה-buffer של master: האופטימיזציה הניחה layout קבוע של 7 שדות-ג'וב בלי עמודת time offsets — הנחות ששתיהן שגויות אצלנו. התאמת ה-buffer ל-layout שלנו הייתה שינוי מסוכן בקוד מחקרי רגיש תמורת רווח זניח (ה-hstack רץ פעם בצעד; צוואר הבקבוק האמיתי הוא הסימולציה, שדווקא כן קיבלה את ההאצות של master).

נאמנות מוחלטת לענף, כולל המוזרויות שלו: למשל, עמודת `can_schedule_now` ב-`_normalize_jobs` נשארת אפס — כך היה בענף המקורי (הפיצ'ר קיים ב-`JobState` אך מעולם לא נורמל לתצפית של `CompactRm`). במהלך העבודה אף נוספה שורה שממלאת אותו — **והוסרה במכוון** אחרי בדיקה מול המקור, כדי לא להזריק שינוי התנהגות שקט. זו נקודת ניסוי מתועדת לפרק 9.

קונפליקט יחיד אך קריטי: איך אוספים את "אירועי העתיד הקרוב" לבניית ה־state.

— **master** שאלתה ממוטבת אחת לחלון זמן קבוע:

```
for e in self.job_events.events_between(self.current_time,
                                       self.current_time + timesteps):
    near_future[e.time - self.current_time].append(e)
```

— **smdp** כש־ `smdp=True`: אוסף את `timesteps` **מועדי־האירועים הבאים** (ללא תלות במרחק בזמן!), עם ריפוד/קיטום לגודל קבוע. זה לב הייצוג: הסוכן רואה את k האירועים הבאים ואת המרחק אליהם, לא חלון זמן קבוע. **ההכרעה:** שילוב לפי מסלול —

```
if smdp:
    # verbatim smdp-branch logic: next `timesteps` distinct event times,
    # padded / trimmed to a fixed size (kept as-is, including its quirks)
    for e in self.job_events:
        ...
else:
    # master's optimized query for the plain-MDP path
    for e in self.job_events.events_between(current, current + timesteps):
        ...
```

למה נשמר גם הריפוד "המוזר": בלולאת הריפוד של הענף יש התנהגות חשודה (מפתחות `last_time + i` כש־ `last_time+1` כבר מתחיל מ־, וריפוד ברשימה ריקה). לא "תיקנתי" — זה קוד שרץ תחת 98 טסטים ירוקים בענף, וכל שינוי בו משנה תצפיות. עדיף לתקן מתישהו במודע, עם ניסוי, ולא כתופעת לוואי של מיזוג. בדיקה נוספת שנעשתה: ה־iteration על `job_events` ב־`EventQueue` החדש של `master` מחזיר רק אירועי עתיד ממוינים — ולכן ה־guard הישן `if e.time < current_time: continue` נשאר תקף (ולא מזיק).

schedgym/test_schedgym.py

- **TOTAL_JOBS:** 122,060 (שלנו) ולא 122,052 (master) — תוצאה ישירה של בחירת הפרסינג הסלחני. הטסט חייב להיות עקבי עם הפרסר.
- **טסט ה־SJF:** `master` הוסיף helper שמחלץ את זמני הג'ובים מתוך וקטור התצפית לפי `offset` מחושב — חישוב שמניח layout ללא עמודות ה־time offsets שלנו, כלומר **היה קורא את התצפית שלנו לא נכון**. נשמר הטסט של הענף, שבדוק מול ה־scheduler ישירות (`env.scheduler.sjf_action`) ולכן עמיד לשינויי layout; ה־helper של `master` נמחק. הטסט עצמו פורט ל־API החדש (ראו פרק 5).

קבצים קטנים

- **deeprn-agent.py:** נשמר ה־guard של Windows על `os.getuid()` (הקומיט 902df95); `master` החזיק את הגרסה היוניקסאית בלבד.
- **__init__.py:** `from envs import ...` בלבד; אצלנו נשמר גם `__version__` כדי `import schedgym` לבדו ירשום את הסביבות ב־`gymnasium` (ההתנהגות שהסקריפטים שלנו מסתמכים עליה). אגב כך תוקן `__all__` שהחזיק מחלקות במקום מחרוזות.
- **.gitignore:** איחוד — הרשימה שלנו הייתה `superset` (`./checkpoint-run1`, `./env`, `pyd.*`).

- `setup.py` **נמחק** יחד עם `setup.cfg` ו-`requirements.txt` — הוחלפו ב-`pyproject.toml` של `master`. תוכן ה-`setup.py` שלנו (רשימת חבילות) מכוסה על-ידי `maturin`.

5. הפורט ל-API של gymnasium

gym 0.21 ו-gymnasium נבדלים בחוזה הבסיסי של סביבה. כל הקוד, הטסטים והסקריפטים פורטו:

gym 0.21 (ישן)	gymnasium (חדש)
reset	obs, info = env.reset(seed=..., options=...)
step	obs, reward, terminated, truncated, info
זריעה	env.seed(n) — reset(seed=n) אצלנו מגושר ל-seed() הישן
גישה לסביבה	gymnasium.make(...).unwrapped לגישה ל-env.scheduler וכד'

5.1 eval_policy.py ו-eval_baselines.py

מעבר ל-`import gymnasium as gym`, פורמטי reset/step חדשים ו-`.unwrapped`. בנוסף, master שינה את ה-raw state של DeepRmEnv מ-4 רכיבים מקובצים ל-6 רכיבים שטוחים (מעבדים וזיכרון בנפרד). כדי לא לשכתב את לוגיקת ההיוריסטיקות, נוספה פונקציה שמקבצת חזרה לצורה שהן מכירות:

```
def split_observation(observation):
    """Groups the raw observation into (current, wait, backlog, time)."""
    if len(observation) == 6:
        c_procs, c_mem, w_procs, w_mem, backlog, time = observation
        return (np.stack((c_procs, c_mem)), np.stack((w_procs, w_mem)),
                backlog, time)
    current, wait, backlog, time = observation # ignore_memory=True case
    return current[None], wait[None], backlog, time
```

np.stack משחזר בדיוק את הצורה הישנה (T, R, 2) / (slots, T, R, 2), כך שחישובי ה-SJF ההיוריסטי נשארו זהים ביט-לביט.

6. תיקוני הפוסט־מיזוג — קומיט 6ce0354

אחרי המיזוג, 178 מתוך 181 טסטים עברו מיד. שלושת הכשלים היו כולם **טסטים חדשים של master שמקודדים את הייצוג שלו** — ולכן תוקנו **הטסטים ולא הקוד**. פירוט מלא, כי כאן נדרש שיקול דעת:

6.1 `test_state_shape_is_valid : len(state) — 2 או 3`

הטסט ציפה ל־ `[processors, memory]` . אצלנו `Scheduler.state` מחזיר **תמיד** גם את עמודת ה־`time offsets` — `[offsets, processors, memory]` — גם במצב MDP (העמודה פשוט מכילה אז `T-1..0`). זו התנהגות הענף המקורית. הציפייה עודכנה ל־3 עם תיעוד.

6.2 `TypeError : test_build_current_state_shape`

הטסט קרא `env.build_current_state(state)` על ה־`state` המלא — ואצלנו האיבר הראשון הוא טאפל של מספרים (`offsets`), לא משאב, ומכאן `'int' object is not subscriptable`. התיקון משקף בדיוק את מה שהקוד האמיתי עושה (`_convert_state`) קורא `current[1:]`: הטסט עודכן ל־ `build_current_state(state[1:])`.

6.3 גבולות התצפית של CompactRm — ממצא מעניין

שני טסטים דרשו שכל ערכי התצפית יהיו ב־`[0,1]`. בפועל נמדדו **-0.329** ו־**1.016**. חקירה העלתה ששניהם **היו קיימים גם בענף הישן** — פשוט אף אחד לא בדק (וה־`Box` הוצהר כ־`[0,1]` באופן לא מדויק):

- **הערך השלילי:** סלוט ג'וב ריק מרופד ב־`sentinel` של `-1`; הטרנספורמציה $(\text{sign}(x) \cdot \log(|x|+e) - 1 + 1) / (\log(\text{time_limit})+1)$ ממפה אותו ל־ ≈ -0.329 .
- **הערך < 1 :** ה־`offsets` הלוגריתמיים — $\log(\text{offset}+1)/\log(\text{time_limit})$ — עוברים את 1 כשאירוע עתידי רחוק מ־`time_limit` (קורה טבעית במצב SMDP שמדלג רחוק).

ההכרעה: לא לשנות את התצפיות (זה היה משנה את מה שהסוכנים ראו עד כה), אלא ליישר את ההצהרות עם המציאות:

```
# compact_env.py – the Box now tells the truth:
self.observation_space = gymnasium.spaces.Box(
    low=-1.0, high=np.inf, shape=((len(self.state)),), dtype=np.float32
)
```

הטסטים בודקים עכשיו טווח שפוי וסופיות (`min ≥ -1`, `isfinite`, `max ≤ 2`) במקום `[0,1]` שרירותי. רווח צדדי: אזהרות ה־`PassiveEnvChecker` של `gymnasium` על "obs לא בתוך ה־space" נעלמות מהצד הזה.

7. שינויי הסביבה

7.1 מה הותקן ולמה

רכיב	גרסה	למה
uv	0.11.28	מנהל הסביבה של upstream; מתקין הכול מ־uv.lock בפקודה אחת
rustup / cargo	stable-msvc / 1.97	ה־build של הפרויקט הוא maturin — קומפילציית ההרחבה schedgym._schedgym_rs דורשת Rust
Python	3.13 (ננעל ב־python. (version	uv בחר תחילה 3.10 מקומי; ננעל 3.13 כדי שהסביבה תהיה באמת עדכנית (upstream כבר הסיר את 3.10 מה־CI שלו)
gymnasium / numpy	2.5.0 / 1.3.0	מגיעים מה־lock של upstream
torch + tensorboard	cpu / 2.21+2.13.0	נדרשים ל־deepm-agent ול־eval_policy; הוכנסו כקבוצת תלויות אופציונלית r1 כדי לא לכפות אותם על כל התקנה

7.2 vendoring של parallelworkloads

pyproject של upstream מושך את parallelworkloads (גנרטורים של Tsafir05/Lublin99, C++/Cython) מ־git — אבל המקור של upstream לא מתקמפל ב־Windows: MSVC חסר את משפחת הפונקציות drand48 (POSIX). התיקון שפיתחנו — rand48_compat.h, מימוש LCG 48-ביט תואם־ספק שמופעל רק תחת #ifdef _WIN32/_MSC_VER ולכן זה־הביט ללינוקס — עדיין לא מוזג ל־upstream, ואין לנו fork ציבורי שלו.

ההחלטה: להכניס את העותק המתוקן לריפו תחת vendor/parallelworkloads (מקור בלבד, בלי ארטיפקטים) ולהפנות אליו:

```
# pyproject.toml
[tool.uv.sources]
# Vendored copy of renatolfc/parallelworkloads with the rand48 POSIX shim
# required to build under MSVC on Windows (upstream PR still pending).
parallelworkloads = { path = "vendor/parallelworkloads" }
```

למה vendoring ולא נתיב מקומי חיצוני: הריפו נהיה עצמאי — clone חדש בכל מכונה (כולל לינוקס, בזכות ה־ifdef) עובד בלי תלות בתיקייה פרטית בשולחן העבודה. כשה־PR יתקבל ב־upstream אפשר יהיה להחזיר את המקור ל־git URL בשורה אחת.

7.3 ההקמה החדשה (ENVIRONMENT.md נכתב מחדש)

```
uv sync --extra test # core + test, builds Rust ext + parallelworkloads
uv sync --extra test --group rl # + torch, tensorboard
```

לעומת 7~ צעדים תלוי־סדר עם נעילות ידניות בסטאק הישן. התיעוד הישן נשמר בהיסטוריית git, ו־requirements־lock.txt נשאר כצילום מצב של הסביבה הישנה לשחזור מדויק אם יידרש.

8. אימות התוצאה

בדיקה	תוצאה
uv run pytest schedgym	passed 181 (98 הטסטים ההיסטוריים של הענף + 83 החדשים של master, מותאמים)
eval_baselines.py -- episodes 3	SJF — 24.4±2.9, Random: -3210±559 · 20.6±4.4 slowdown, -2600±570 תגמול SJF: עדיף, כמצופה (בדיקת שפיות סמנטית)
Rust backend	schedgym._schedgym_rs נבנה ונטען; Heap.__module__ == 'schedgym._schedgym_rs'
torch	cpu+2.13.0 מיובא תקין; eval_policy.py ו-deeprm-agent.py מתקמפלים

מה המשמעות של "181 ירוקים" כאן

חבילת הטסטים ההיסטורית של הענף מקודדת את התנהגות ה-SMDP (ייצוגים, SJF, פרסינג, 122,060 ג'ובים). העובדה שהיא עוברת ללא שינוי סמנטי — רק פורט API — היא הראיה המרכזית לכך שהמיזוג לא שינה את המחקר.

9. נקודות פתוחות והמלצות

1. `can_schedule_now` לא מגיע לתצפית של `CompactRm`. השדה מחושב וממולא ב-`JobState` אך העמודה שלו נשארת אפס ב-`_normalize_jobs` — כך היה בענף המקורי ונשמר במכוון. חשיפתו לסוכן היא שורה אחת (`_sumdiv(ret[i], 7, job.can_schedule_now, 1)`) ומועמדת טבעית לניסוי מבוקר.
2. **checkpoints ישנים (pth):** פריסת התצפית של `CompactRm` לא השתנתה ולכן צפויה תאימות, אבל זה לא אומת בפועל — מומלץ להריץ `eval_policy` על checkpoint ישן לפני שמסתמכים.
3. **תרומת ה-shim ל-upstream:** `rand48_compat.h` ראוי ל-PR ל-`renatolfc/parallelworkloads`; קבלתו תאפשר לוותר על ה-`vendoring`.
4. **הריפוד ב-Scheduler.state במצב SMDP** (סעיף 4.3) חשוד כבאגי אך הושאר — אם משפרים, לעשות זאת כניסוי מדיד ולא כתיקון אגבי.
5. **אזהרות dtype:** `gymnasium` מזהיר ש-`obs` מוחזר כ-`float64` בעוד ה-`Box` מוצהר כ-`float32` — קוסמטי, לא נגענו כדי לא לשנות דיוק חישובי; אפשר ליישר בהמשך.